dlorg

Purpose with this lab is to use your knowledge of Linux filesystem, file processing and bash scripting to do some real automations to organize files for you. A system administor really loves to keep stuffs tidy and would rather have scripts to automatically handle that rather than manually tidying up stuffs.



Figure 1: image generated from nano banana 2

Backgrund

You are on an interview for a Linux sysadmin role and have come past the initial HR interview step. Now you have given a technical task to solve a common problem that many employees have.

It is not uncommon that they have downloaded too many files that so that their downloads folder looks something like this

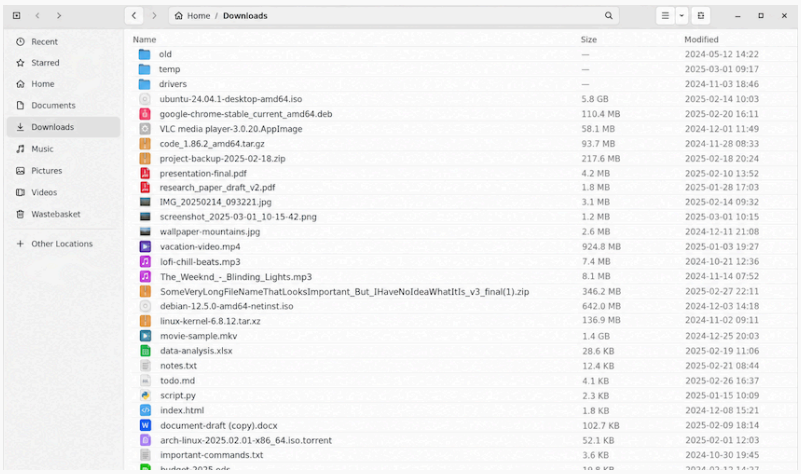


Figure 2: image generated from gpt-image 2.5

This is really messy and looking for a file takes forever, which obviously is time they don't want to spend looking for files.

Congratulations, you have given the task to clean up this mess.

Tasks

The interview assignment consists of the following tasks below.

Task 0 - Warmup

In this task you get to build the skeleton of your lab

a) create a new repo for your lab called **dlorg_firstname_lastname**, then clone it
b) create a .gitignore file and insert the content from here to ignore vim related stuffs
c) create a bash script called dlorg
d) give suitable permissions for it

> ℹ it's important to create several meaningful commits throughout the lab so that you show that you understand git and github

Task 1 - the downloads organizer

When there is a new file, a file change name or a file is moved into your Downloads directory, then that particular file should be automatically moved into an appropriate directory in Downloads. For example it could look like this

```
$ touch {a,b}.txt cool{1..3}.pdf awesome_{1..3}.mp4 beautiful.jpg
coolu.png notes.docx
$ tree
├── docs
│   └── notes.docx
├── images
│   ├── beautiful.jpg
│   └── coolu.png
├── other
│   ├── awesome_1.mp4
│   ├── awesome_2.mp4
│   └── awesome_3.mp4
├── pdfs
│   ├── cool1.pdf
│   ├── cool2.pdf
│   └── cool3.pdf
└── text
    ├── a.txt
    └── b.txt
```

Do come up with some more than these directories.

Task 2 - Testing your script

Now when you have your script, the time has come to test it out.

a) Use `tmux` to divide your terminal so you have one shell running the script and one inside of Downloads
b) Create some files like in task 1 and control that they get into right folders
c) use `mv` to move some files into Downloads directory and check where they go

d) **BONUS**: handle the case that a directory is deleted, e.g. delete images directory and then create a .png file
e) **BONUS**: move in some files from your host machine into the Downloads directory and check where they go

Task 3 - Make it into a service (BONUS)

Now that your script works as expected, you don't want to always open a new shell just to run this script. Instead you want it to run at startup of your system as a background service that always monitors the Downloads directory.

Task 4 - Documentation

Document `dlorg` and its capabilities in `README.md`. Don't make it too long but include relevant screenshots and explanations. Think of it as a guide for someone that is gonna use your program.

Task 5 - Video explanation

Now it is time to explain how you created this script step by step. The audience are employees in the IT department, but they also benefit from interesting presentation. Showcase the capabilities of your script and then moving on to how it works and show some testing.

Also if you have done the BONUS parts, show them as well in the video.

The video should be between 5-10 minutes where you share screen and present.

Handin

Send the following to student portal

- a link to your github repo
- a link to your video, make sure to set the permission so that everyone with the link can access it

LLM usage

You can use LLM to code some smaller parts, but not solve entire tasks. It is important that you understand the code and can explain it. You can use LLM as a thought partner.

Make a note where you use LLM.

Grading

Godkänt

- done all tasks except those marked with **BONUS**
- can explain what you have done in the video

Väl Godkänt

- also solved the **BONUS** tasks
- solved the tasks on higher level than for G
- explanations in video have technical depth with good use of Linux related terminologies
- code should be concise, clean, readable and follow DRY (don't repeat yourself)